

Step 4 — A Fallback When a Server Is Unreachable, and a Full Working Demo — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below build on Step 3's `notes_server.py` / `filesystem_server.py` and `servers_config.py`, with one entry now deliberately broken to prove the fallback path:

```
# servers_config.py
SERVERS = [
    {"name": "notes", "command": "python", "args": ["notes_server.py"]},
    {"name": "filesystem", "command": "python", "args": ["this_file_does_not_exist.py"]}
]
```

Basic Version

Approach 1 — the direct way

```
# server_pool.py
import asyncio
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from tool_bridge import call_mcp_tool

NAMESPACE_SEP = "__"

class MCPServerPool:
    def __init__(self):
        self._sessions = {}
        self._routing = {}
        self._all_tools = []
        self._dead_servers = set()
        self._stack = AsyncExitStack()
```

```

async def connect_all(self, servers_config, connect_timeout=10.0):
    for server in servers_config:
        try:
            params = StdioServerParameters(command=server["command"], args=se
            read, write = await asyncio.wait_for(
                self._stack.enter_async_context(stdio_client(params)), timeou
            )
            session = await asyncio.wait_for(
                self._stack.enter_async_context(ClientSession(read, write)),
            )
            await asyncio.wait_for(session.initialize(), timeout=connect_time
        except Exception as e:
            print(f"[{server['name']}] unreachable: {type(e).__name__}: {e} -
            continue

        self._sessions[server["name"]] = session
        tools_result = await session.list_tools()
        for tool in tools_result.tools:
            namespaced_name = server["name"] + NAMESPACE_SEP + tool.name
            self._routing[namespaced_name] = (server["name"], tool.name)
            self._all_tools.append({
                "type": "function",
                "function": {
                    "name": namespaced_name,
                    "description": tool.description or "",
                    "parameters": tool.inputSchema,
                },
            })
        return self.get_current_tools()

def get_current_tools(self):
    return [
        t for t in self._all_tools
        if self._routing[t["function"]["name"]][0] not in self._dead_servers
    ]

async def call_tool(self, namespaced_name, arguments):
    server_name, original_name = self._routing[namespaced_name]
    if server_name in self._dead_servers:
        return f"Error: the '{server_name}' server is unavailable for the res
    session = self._sessions[server_name]
    try:
        return await call_mcp_tool(session, original_name, arguments)
    except Exception as e:
        self._dead_servers.add(server_name)
        return f"Error: the '{server_name}' server is not responding ({type(e)

```

```
async def close(self):  
    await self._stack.aclose()
```

Expected output connecting to `SERVERS` above:

```
[filesystem] unreachable: FileNotFoundError: [Errno 2] No such file or directory:
```

followed by a pool that only has `notes__search_notes` and `notes__read_note` available — the `filesystem` entry failed, but `notes` connected and works normally.

This works correctly for the connect-time failure. It doesn't yet check for a duplicate server name (Step 3's Advanced version had this — keep it, it's omitted here only to keep this listing focused on what's new), and `agent.py` still needs updating to call `pool.get_current_tools()` fresh each loop instead of reusing one list — shown next.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `agent.py` updated to re-check available tools every iteration

```
# agent.py
import json
from openai import OpenAI

client = OpenAI()

class MaxIterationsExceeded(Exception):
    def __init__(self, message, log):
        super().__init__(message)
        self.log = log

async def run_agent(task, pool, max_iterations=6):
    messages = [
        {"role": "system", "content": "You are a helpful assistant. Use the available tools to complete the task."},
        {"role": "user", "content": task},
    ]
    log = []

    for step in range(max_iterations):
        current_tools = pool.get_current_tools() # <-- fresh every step,
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=current_tools,
        )
        message = response.choices[0].message

        if not message.tool_calls:
            log.append({"step": step, "type": "final_answer", "content": message.content})
            return {"answer": message.content, "log": log}

        messages.append(message)
        for tool_call in message.tool_calls:
            name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)
```

```

        log.append({"step": step, "type": "tool_call", "tool": name, "argumen

        result_text = await pool.call_tool(name, arguments)
        log.append({"step": step, "type": "observation", "tool": name, "resul

        messages.append({"role": "tool", "tool_call_id": tool_call.id, "conte

        raise MaxIterationsExceeded(f"Agent did not finish within {max_iterations} st

```

Expected output: on a run where the `filesystem` server never connected at all, `current_tools` only ever contains `notes__*` entries -- the model is never even offered a tool it can't reach, from the very first step, because `get_current_tools()` already filtered it out during `connect_all`.

Difference from Basic: `run_agent`'s signature changes from taking a fixed `openai_tools` list to taking the `pool` itself, and it asks the pool for `get_current_tools()` on every single iteration -- the one change that makes a server dying *mid-task* (not just one that never connected) actually disappear from what the model is offered on the very next step, instead of staying visible until the whole run ends.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the final `main.py` demo

```
# main.py
import asyncio
from servers_config import SERVERS
from server_pool import MCPServerPool
from agent import run_agent, MaxIterationsExceeded

async def main():
    pool = MCPServerPool()
    try:
        available_tools = await pool.connect_all(SERVERS)
        print(f"Connected. {len(available_tools)} tools available: "
              f"{[t['function']['name'] for t in available_tools]}\n")

        tasks = [
            "Check my notes for anything about the deploy process, and also check",
            "if there's a deploy script in the project folder.",
            "Look up something in my notes about onboarding.",
        ]
        for task in tasks:
            print(f"--- Task: {task} ---")
            try:
                result = await run_agent(task, pool)
                print(f"Answer: {result['answer']}\n")
            except MaxIterationsExceeded as e:
                print(f"Gave up after too many steps: {e}\n")
    finally:
        await pool.close()

if __name__ == "__main__":
    asyncio.run(main())
```

Expected output, with `filesystem` deliberately broken (as in `servers_config.py` above):

```
[filesystem] unreachable: FileNotFoundError: ... -- continuing without it
Connected. 2 tools available: ['notes__search_notes', 'notes__read_note']

--- Task: Check my notes for anything about the deploy process, and also check if
Answer: I found a note about the deploy process (...). I wasn't able to check the

--- Task: Look up something in my notes about onboarding. ---
Answer: ...
```

The first task's answer explicitly says it couldn't check the filesystem -- that's the agent reacting correctly to a tool it was never even offered, not silently pretending the filesystem half of the task didn't exist.

Approach 2 — `test_agent.py`, proving the mid-task drop path without needing to actually kill a process

```
# test_agent.py
import asyncio
from servers_config import SERVERS
from server_pool import MCPServerPool
from agent import run_agent

async def raise_connection_lost(*args, **kwargs):
    raise ConnectionError("simulated: the notes server just disappeared")

async def test_mid_task_server_drop():
    pool = MCPServerPool()
    try:
        await pool.connect_all(SERVERS)
        assert "notes" not in pool._dead_servers, "test setup assumes notes conne

        # simulate the notes server dying AFTER a successful connection,
        # by breaking its session's call_tool for the rest of this test
        broken_session = pool._sessions["notes"]
        broken_session.call_tool = raise_connection_lost

        result = await run_agent("Search my notes for anything about deploys.", p

        assert "notes" in pool._dead_servers, "pool should have marked 'notes' de
        remaining_names = [t["function"]["name"] for t in pool.get_current_tools(
        assert not any(name.startswith("notes__") for name in remaining_names), \
            "notes tools should no longer be offered after it went dead"
        print("PASS: agent survived a mid-task server drop and stopped offering i
```

```

        print(f"Final answer: {result['answer']}")
    finally:
        await pool.close()

if __name__ == "__main__":
    asyncio.run(test_mid_task_server_drop())

```

Expected output:

```

PASS: agent survived a mid-task server drop and stopped offering its tools.
Final answer: I wasn't able to search your notes just now -- that tool became una

```

Difference from Intermediate, and between these 2 Advanced pieces: Intermediate proves the pieces work in isolation. Approach 1 is the actual promised deliverable -- one script, run end to end, that degrades gracefully with a real broken server in its config. Approach 2 is the test that actually exercises the *harder* failure mode named in the README (a server that drops **after** connecting successfully, not one that was never reachable at all) -- without needing the fragility of spawning and then forcibly killing a real subprocess mid-test, which would make this test flaky across machines for no real benefit over simulating the same failure at the session level.

Which one should you actually write? All of it, in this order: the resilience additions to `server_pool.py` and `agent.py` (Basic/Intermediate above) are the actual point of this step and aren't optional. Approach 1's `main.py` is your real, portfolio-facing deliverable -- the thing you'd actually show someone. Approach 2's test is what proves Step 4 solved the problem it claims to solve, rather than just looking like it does on the one happy-path run you tried by hand -- keep it in the repo, it's cheap to run and it's the difference between "I think this handles failures" and "I have a test that proves it."